MODUL PRAKTIKUM MINI PROJECT:
PENERAPAN MVC & OOP SEDERHANA

MVC Native, Pemrograman Berorientasi Objek (OOP) & Keamanan Web Dasar

Studi Kasus Sederhana: Manajemen Data Pengguna / Operator Sistem (1 Tabel Tunggal)

Mata Kuliah	Pemrograman Web Dasar	**Institusi**	STMIK IKMI Cirebon
Dosen Pengampu	Rosidin, S. Kom., M. Kom.	**Semester / TA**	III (Tiga) / Ganjil 2026–2027
Model Arsitektur	MVC Native Sederhana	**Basis Database**	MySQL 8.0 & PDO Extension
Fitur Inti	CRUD Lengkap	**Fokus Keamanan**	Anti SQL Injection & Anti XSS

BAB 1: PENGANTAR SEDERHANA: MEMAHAMI POLA MVC & OOP DALAM PENGEMBANGAN WEB

Dalam rekayasa perangkat lunak dan pengembangan aplikasi web modern, kerapian struktur kode program merupakan pondasi utama. Pada awal belajar pemrograman PHP, pemula sering menuliskan seluruh instruksi (koneksi database, query SQL, proses formulir, dan kode HTML) bercampur aduk dalam satu berkas tunggal (dikenal dengan istilah 'Spaghetti Code'). Meskipun aplikasi dapat berjalan, cara penulisan tersebut memiliki banyak kelemahan: kode sulit dibaca, memicu duplikasi logika, dan menyulitkan pelacakan sumber error saat sistem berkembang.

Pola **Model-View-Controller (MVC)** dan **Pemrograman Berorientasi Objek (OOP)** memecah tanggung jawab program ke dalam 3 komponen yang terpisah secara tegas (*Separation of Concerns*):

> 📌 **Analogi Restoran / Perusahaan Layanan untuk Memahami MVC**
> • Model (Dapur Pengolah Data / Database): Bertanggung jawab murni terhadap penyimpanan, pengambilan, dan manipulasi data di database MySQL. Model sama sekali tidak mengurusi desain tampilan atau warna tombol.
> • View (Penyajian / Antarmuka Layar): Bagian presentasi visual yang dilihat langsung oleh pengguna di browser (berisi HTML, CSS, dan tabel data). View hanya menampilkan data yang diteruskan kepadanya.
> • Controller (Pelayan / Pengatur Lalu Lintas): Bertindak sebagai perantara yang menerima permintaan pengguna (klik tombol / kirim form), meminta Model mengolah data, lalu meneruskan hasilnya ke View yang sesuai.

Pada modul praktikum pengantar ini, **fungsi login sengaja ditiadakan** agar mahasiswa dapat fokus memahami alur komunikasi data antara Controller, Model, dan View serta menguasai siklus manipulasi data (CRUD) secara tuntas dan mandiri.

BAB 2: SKEMA BASIS DATA 1 TABEL (`users`)

Aplikasi ini menggunakan **1 tabel tunggal** bernama `users` dengan 5 kolom sederhana untuk mencatat data pengguna atau operator sistem informasi:

[SQL DATABASE] Berkas : `database.sql` *(Skema 1 Tabel 5 Kolom)*

```sql
-- 1. Buat Database Latihan
CREATE DATABASE IF NOT EXISTS `db_web_dasar`;
USE `db_web_dasar`;
```

```sql
-- 2. Buat 1 Tabel Tunggal: users (Hanya 5 Kolom Sederhana)
CREATE TABLE IF NOT EXISTS `users` (
    `id` INT AUTO_INCREMENT PRIMARY KEY,         -- Kolom 1: ID Unik Pengguna
    `username` VARCHAR(50) NOT NULL UNIQUE,      -- Kolom 2: ID Akun Pengguna
    `nama_lengkap` VARCHAR(100) NOT NULL,        -- Kolom 3: Nama Lengkap Pengguna
    `role` VARCHAR(50) NOT NULL,              -- Kolom 4: Peran / Hak Akses Pengguna
    `email` VARCHAR(100) NOT NULL             -- Kolom 5: Alamat Email Resmi
) ENGINE=InnoDB;

-- 3. Data Awal Pengguna Sistem
INSERT INTO `users` (`username`, `nama_lengkap`, `role`, `email`) VALUES
('admin01', 'Dewi Lestari', 'Administrator', 'dewi.admin@perusahaan.com'),
('operator01', 'Ahmad Fauzi', 'Operator Staf', 'ahmad.fauzi@perusahaan.com'),
('spv01', 'Siti Maryam', 'Supervisor', 'siti.maryam@perusahaan.com');
```

BAB 3: STRUKTUR DIREKTORI PROYEK SEDERHANA

Buka Visual Studio Code dan buat folder baru bernama `mini_project`. Susunlah berkas dan folder sebagai berikut:

[STRUKTUR FOLDER] **Berkas :** **Susunan Berkas Proyek** *(7 Berkas Sederhana)*

```
mini_project/
│
├── config/
│   └── Database.php         <- Kelas koneksi PDO (Anti SQL Injection)
│
├── models/
│   └── User.php             <- Model CRUD: getAll, getById, create, update,
delete
│
├── controllers/
│   └── UserController.php   <- Controller CRUD: index, create, edit, delete
│
├── views/
│   ├── index.php          <- Tampilan tabel data pengguna (CSS sederhana & rapi)
│   ├── create.php           <- Formulir tambah data pengguna baru
│   └── edit.php             <- Formulir ubah data pengguna
│
├── database.sql             <- Script SQL basis data
└── index.php              <- Pintu masuk aplikasi (Front Router 17 baris)
```

BAB 4: LANGKAH 1 - KONEKSI DATABASE PDO AMAN (`config/Database.php`)

Pustaka PDO (PHP Data Objects) digunakan untuk menghubungkan aplikasi ke database MySQL secara aman. Konfigurasi `PDO::ATTR_EMULATE_PREPARES => false` memastikan kueri dikompilasi secara terpisah dari parameter data, sehingga menutup celah SQL Injection secara mutlak.

[OOP KONEKSI] **Berkas :** **config/Database.php** *(Koneksi PDO Strict Mode)*

```php
<?php
// config/Database.php - Kelas Koneksi PDO Aman
class Database {
    private static $host = 'localhost';
    private static $db   = 'db_web_dasar';
    private static $user = 'root';
    private static $pass = '';
```

```
    private static $pdo  = null;

    public static function connect() {
        if (self::$pdo === null) {
            try {
                $dsn = 'mysql:host=' . self::$host . ';dbname=' . self::$db .
';charset=utf8mb4';
                self::$pdo = new PDO($dsn, self::$user, self::$pass, [
                    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES   => false // MENCEGAH SQL
INJECTION MUTLAK
                ]);
            } catch (PDOException $e) {
                die('Koneksi basis data gagal: ' . $e->getMessage());
            }
        }
        return self::$pdo;
    }
}
```

BAB 5: LANGKAH 2 - MEMBUAT MODEL OOP LENGKAP (`models/User.php`)

Model adalah tempat berkumpulnya fungsi pengolah data tabel `users`. Model memuat 5 method CRUD lengkap: 1) `getAll()` untuk Read Semua Data, 2) `getById($id)` untuk Read Satu Data, 3) `create(...)` untuk Tambah Data, 4) `update(...)` untuk Ubah Data, dan 5) `delete($id)` untuk Hapus Data. Seluruh parameter input dieksekusi dengan **Prepared Statements**.

[MODEL CRUD] Berkas : models/User.php *(CRUD Lengkap & Prepared Statements)*

```php
<?php
// models/User.php - Model CRUD Pengguna Sistem
require_once __DIR__ . '/../config/Database.php';

class User {
    private $db;

    public function __construct() {
        $this->db = Database::connect();
    }

    // [READ ALL] Mengambil semua pengguna untuk ditampilkan di tabel
    public function getAll() {
        $stmt = $this->db->query('SELECT * FROM users ORDER BY id DESC');
        return $stmt->fetchAll();
    }

    // [READ ONE] Mengambil 1 baris data pengguna berdasarkan ID untuk form edit
    public function getById($id) {
        $stmt = $this->db->prepare('SELECT * FROM users WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch();
    }

    // [CREATE] Menambah baris data pengguna baru
    public function create($username, $nama, $role, $email) {
        $sql = 'INSERT INTO users (username, nama_lengkap, role, email)
                VALUES (:username, :nama, :role, :email)';
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([
```

```
            ':username' => $username,
            ':nama'     => $nama,
            ':role'     => $role,
            ':email'    => $email
        ]);
    }

    // [UPDATE] Memperbarui data pengguna berdasarkan ID
    public function update($id, $username, $nama, $role, $email) {
        $sql = 'UPDATE users SET username = :username, nama_lengkap = :nama,
                role = :role, email = :email WHERE id = :id';
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([
            ':id'       => $id,
            ':username' => $username,
            ':nama'     => $nama,
            ':role'     => $role,
            ':email'    => $email
        ]);
    }

    // [DELETE] Menghapus data pengguna berdasarkan ID
    public function delete($id) {
        $stmt = $this->db->prepare('DELETE FROM users WHERE id = :id');
        return $stmt->execute([':id' => $id]);
    }
}
```

BAB 6: LANGKAH 3 - MEMBUAT CONTROLLER (`controllers/UserController.php`)

Controller bertindak sebagai pengatur alur logika. Controller memanggil Model untuk mengambil atau menyimpan data, lalu memanggil View yang sesuai tanpa memerlukan mekanisme otentikasi login.

[CONTROLLER] Berkas : controllers/UserController.php *(Logika CRUD Lengkap Tanpa Login)*

```php
<?php
// controllers/UserController.php - Pengatur Alur CRUD Pengguna
require_once __DIR__ . '/../models/User.php';

class UserController {
    private $model;

    public function __construct() {
        $this->model = new User();
    }

    // 1. Tampilkan Halaman Tabel Utama
    public function index() {
        $users = $this->model->getAll();
        require_once __DIR__ . '/../views/index.php';
    }

    // 2. Formulir & Proses Tambah Data
    public function create() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $username = trim($_POST['username'] ?? '');
            $nama     = trim($_POST['nama_lengkap'] ?? '');
```

```
            $role     = trim($_POST['role'] ?? '');
            $email    = trim($_POST['email'] ?? '');

            if (!empty($username) && !empty($nama)) {
                $this->model->create($username, $nama, $role, $email);
                header('Location: index.php');
                exit;
            }
        }
        require_once __DIR__ . '/../views/create.php';
    }

    // 3. Formulir & Proses Edit Data
    public function edit() {
        $id = (int)($_GET['id'] ?? 0);
        $user = $this->model->getById($id);
        if (!$user) {
            die('Data pengguna tidak ditemukan!');
        }

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $username = trim($_POST['username'] ?? '');
            $nama     = trim($_POST['nama_lengkap'] ?? '');
            $role     = trim($_POST['role'] ?? '');
            $email    = trim($_POST['email'] ?? '');

            if (!empty($username) && !empty($nama)) {
                $this->model->update($id, $username, $nama, $role, $email);
                header('Location: index.php');
                exit;
            }
        }
        require_once __DIR__ . '/../views/edit.php';
    }

    // 4. Proses Hapus Data
    public function delete() {
        $id = (int)($_GET['id'] ?? 0);
        if ($id > 0) {
            $this->model->delete($id);
        }
        header('Location: index.php');
        exit;
    }
}
```

BAB 7: LANGKAH 4 - MEMBUAT VIEW (TABEL CSS SEDERHANA & ANTI-XSS)

Tampilan antarmuka disajikan dalam format tabel yang bersih menggunakan CSS internal sederhana. Setiap data dinamis wajib disaring dengan fungsi `htmlspecialchars()` agar terbebas dari celah serangan script jahat (Anti XSS).

[VIEW TABEL] Berkas : views/index.php *(Tabel Data dengan CSS Rapi & Anti-XSS)*

```
<!-- views/index.php - Tampilan Tabel Data dengan CSS Sederhana -->
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
```

```
    <title>Manajemen Data Pengguna - Latihan MVC</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Arial, sans-serif; margin: 30px;
background-color: #f4f6f9; color: #333; }
        .card { background: #ffffff; padding: 25px; border-radius: 8px; box-
shadow: 0 2px 6px rgba(0,0,0,0.08); max-width: 950px; margin: auto; }
        h2 { color: #0f2c59; margin-bottom: 5px; }
        p.desc { color: #666; font-size: 14px; margin-bottom: 20px; }
        .btn-add { display: inline-block; padding: 9px 16px; background-color:
#0f2c59; color: #fff; text-decoration: none; border-radius: 4px; font-weight:
bold; font-size: 13px; margin-bottom: 15px; }
        .btn-add:hover { background-color: #184c78; }
        table { width: 100%; border-collapse: collapse; margin-top: 10px; font-
size: 14px; }
        th, td { border: 1px solid #e0e0e0; padding: 10px 12px; text-align: left;
}
        th { background-color: #0f2c59; color: #ffffff; font-weight: 600; }
        tr:nth-child(even) { background-color: #f9fbfd; }
        tr:hover { background-color: #f1f5f9; }
        .badge-role { background-color: #e8f4fd; color: #0f2c59; padding: 4px 8px;
border-radius: 4px; font-size: 12px; font-weight: bold; }
        .btn-edit { background: #f39c12; color: #fff; padding: 5px 10px; text-
decoration: none; border-radius: 3px; font-size: 12px; font-weight: bold; margin-
right: 5px; }
        .btn-delete { background: #e74c3c; color: #fff; padding: 5px 10px; text-
decoration: none; border-radius: 3px; font-size: 12px; font-weight: bold; }
        .btn-edit:hover { background: #d68910; }
        .btn-delete:hover { background: #c0392b; }
    </style>
</head>
<body>
<div class="card">
    <h2>👥 Manajemen Data Pengguna Sistem</h2>
    <p class="desc">Latihan Dasar MVC & OOP: Pengelolaan Data Pengguna / Operator
(CRUD Lengkap Tanpa Login)</p>

    <a href="index.php?action=create" class="btn-add">+ Tambah Pengguna Baru</a>

    <table>
        <thead>
            <tr>
                <th style="width: 50px;">No</th>
                <th>Username</th>
                <th>Nama Lengkap</th>
                <th>Peran / Role</th>
                <th>Email Resmi</th>
                <th style="width: 150px; text-align: center;">Aksi</th>
            </tr>
        </thead>
        <tbody>
            <?php if (empty($users)): ?>
                <tr><td colspan="6" style="text-align: center; color: #999;
padding: 20px;">Belum ada data pengguna.</td></tr>
            <?php else: ?>
                <?php $no = 1; foreach ($users as $u): ?>
                <tr>
                    <td><?= $no++ ?></td>
                    <!-- PENTING: htmlspecialchars() mencegah serangan XSS -->
                    <td><b><?= htmlspecialchars($u['username']) ?></b></td>
                    <td><?= htmlspecialchars($u['nama_lengkap']) ?></td>
                    <td><span class="badge-role"><?= htmlspecialchars($u['role'])
?></span></td>
```

```
                    <td><?= htmlspecialchars($u['email']) ?></td>
                    <td style="text-align: center;">
                        <a href="index.php?action=edit&id=<?= (int)$u['id'] ?>"
class="btn-edit">✏️ Edit</a>
                        <a href="index.php?action=delete&id=<?= (int)$u['id'] ?>"
class="btn-delete" onclick="return confirm('Yakin ingin menghapus pengguna
ini?')">🗑️ Hapus</a>
                    </td>
                </tr>
                <?php endforeach; ?>
            <?php endif; ?>
        </tbody>
    </table>
</div>
</body>
</html>
```

[VIEW CREATE] Berkas : views/create.php *(Formulir Tambah Data)*

```
<!-- views/create.php - Formulir Tambah Pengguna Baru -->
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Tambah Pengguna Baru</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Arial, sans-serif; margin: 30px;
background-color: #f4f6f9; color: #333; }
        .card { max-width: 500px; margin: auto; background: #fff; padding: 25px;
border-radius: 8px; box-shadow: 0 2px 6px rgba(0,0,0,0.08); }
        h3 { color: #0f2c59; margin-bottom: 18px; border-bottom: 2px solid
#0f2c59; padding-bottom: 8px; }
        .form-group { margin-bottom: 15px; }
        label { display: block; font-weight: bold; margin-bottom: 6px; font-size:
13.5px; }
        input, select { width: 100%; padding: 9px; box-sizing: border-box; border:
1px solid #ccc; border-radius: 4px; font-size: 13.5px; }
        input:focus, select:focus { border-color: #0f2c59; outline: none; }
        .btn-save { background: #0f2c59; color: white; border: none; padding: 10px
18px; border-radius: 4px; cursor: pointer; font-weight: bold; font-size: 13.5px; }
        .btn-save:hover { background: #184c78; }
        .btn-cancel { color: #7f8c8d; text-decoration: none; margin-left: 12px;
font-size: 13.5px; }
    </style>
</head>
<body>
<div class="card">
    <h3>➕ Tambah Pengguna Baru</h3>
    <form method="POST" action="index.php?action=create">
        <div class="form-group">
            <label>Username Pengguna</label>
            <input type="text" name="username" placeholder="Contoh: user02"
required>
        </div>
        <div class="form-group">
            <label>Nama Lengkap</label>
            <input type="text" name="nama_lengkap" placeholder="Contoh: Budi
Santoso" required>
        </div>
        <div class="form-group">
            <label>Peran / Role</label>
            <select name="role" required>
                <option value="Administrator">Administrator</option>
```

```
                <option value="Operator Staf">Operator Staf</option>
                <option value="Manajer Operasional">Manajer Operasional</option>
                <option value="Supervisor">Supervisor</option>
            </select>
        </div>
        <div class="form-group">
            <label>Email Resmi</label>
            <input type="email" name="email" placeholder="Contoh:
budi@perusahaan.com" required>
        </div>
        <button type="submit" class="btn-save">💾 Simpan Pengguna</button>
        <a href="index.php" class="btn-cancel">Batal</a>
    </form>
</div>
</body>
</html>
```

[VIEW UPDATE] Berkas : views/edit.php *(Formulir Edit Data)*

```
<!-- views/edit.php - Formulir Ubah Data Pengguna -->
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Edit Data Pengguna</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Arial, sans-serif; margin: 30px;
background-color: #f4f6f9; color: #333; }
        .card { max-width: 500px; margin: auto; background: #fff; padding: 25px;
border-radius: 8px; box-shadow: 0 2px 6px rgba(0,0,0,0.08); }
        h3 { color: #f39c12; margin-bottom: 18px; border-bottom: 2px solid
#f39c12; padding-bottom: 8px; }
        .form-group { margin-bottom: 15px; }
        label { display: block; font-weight: bold; margin-bottom: 6px; font-size:
13.5px; }
        input, select { width: 100%; padding: 9px; box-sizing: border-box; border:
1px solid #ccc; border-radius: 4px; font-size: 13.5px; }
        .btn-update { background: #f39c12; color: white; border: none; padding:
10px 18px; border-radius: 4px; cursor: pointer; font-weight: bold; font-size:
13.5px; }
        .btn-update:hover { background: #d68910; }
        .btn-cancel { color: #7f8c8d; text-decoration: none; margin-left: 12px;
font-size: 13.5px; }
    </style>
</head>
<body>
<div class="card">
    <h3>✏️ Edit Data Pengguna</h3>
    <form method="POST" action="index.php?action=edit&id=<?= (int)$user['id'] ?>">
        <div class="form-group">
            <label>Username Pengguna</label>
            <input type="text" name="username" value="<?=
htmlspecialchars($user['username']) ?>" required>
        </div>
        <div class="form-group">
            <label>Nama Lengkap</label>
            <input type="text" name="nama_lengkap" value="<?=
htmlspecialchars($user['nama_lengkap']) ?>" required>
        </div>
        <div class="form-group">
            <label>Peran / Role</label>
            <select name="role" required>
                <option value="Administrator" <?= ($user['role'] ===
```

```
'Administrator') ? 'selected' : '' ?>>Administrator</option>
                <option value="Operator Staf" <?= ($user['role'] === 'Operator
Staf') ? 'selected' : '' ?>>Operator Staf</option>
                <option value="Manajer Operasional" <?= ($user['role'] ===
'Manajer Operasional') ? 'selected' : '' ?>>Manajer Operasional</option>
                <option value="Supervisor" <?= ($user['role'] === 'Supervisor') ?
'selected' : '' ?>>Supervisor</option>
            </select>
        </div>
        <div class="form-group">
            <label>Email Resmi</label>
            <input type="email" name="email" value="<?=
htmlspecialchars($user['email']) ?>" required>
        </div>
        <button type="submit" class="btn-update">🔄 Simpan Perubahan</button>
        <a href="index.php" class="btn-cancel">Batal</a>
    </form>
</div>
</body>
</html>
```

BAB 8: LANGKAH 5 - PINTU MASUK APLIKASI (INDEX.PHP)

Berkas `index.php` adalah Front Controller sederhana. Berkas ini menerima aksi melalui URL seperti `?action=create` atau `?action=delete`, lalu memanggil method pada `UserController`.

[ROUTER] Berkas : index.php *(Front Router 17 Baris)*

```php
<?php
// index.php - Front Router Sederhana
require_once __DIR__ . '/controllers/UserController.php';

$controller = new UserController();
$action = $_GET['action'] ?? 'index';

if ($action === 'create') {
    $controller->create();
} elseif ($action === 'edit') {
    $controller->edit();
} elseif ($action === 'delete') {
    $controller->delete();
} else {
    $controller->index();
}
```

BAB 9: PANDUAN UJI COBA & EKSPERIMEN KEAMANAN

9.1 Menjalankan Aplikasi di Visual Studio Code

1. Jalankan Service: Pastikan Apache dan MySQL di XAMPP / Laragon sudah dalam keadaan aktif (Running).

2. Impor Database: Buka browser ke http://localhost/phpmyadmin, buat database db_web_dasar, lalu jalankan query skema dari BAB 2.

3. Jalankan Server: Buka folder mini_project di VS Code, buka Terminal (Ctrl + `), lalu ketik: php -S localhost:8000

4. Buka Aplikasi: Buka browser dan buka alamat: http://localhost:8000. Tabel data pengguna langsung muncul tanpa perlu login.

9.2 Pengujian Siklus CRUD Lengkap

• **Uji Tambah (Create):** Klik tombol '+ Tambah Pengguna Baru', isi formulir, klik 'Simpan Pengguna'. Data langsung bertambah di tabel.
• **Uji Tampil (Read):** Perhatikan tabel data yang tersaji rapi dengan warna selang-seling, badge peran, dan efek hover CSS.
• **Uji Ubah (Update):** Klik tombol 'Edit' warna oranye pada salah satu baris, ubah nama lengkap atau peran, simpan, dan periksa pembaruan di tabel.
• **Uji Hapus (Delete):** Klik tombol 'Hapus' warna merah, setujui dialog konfirmasi, dan pastikan baris terhapus dari tabel dan basis data.

9.3 Eksperimen Keamanan Langsung

• **Anti SQL Injection:** Coba masukkan teks: ' OR '1'='1 pada nama lengkap saat menambah data. Hasilnya: Tersimpan sebagai teks harfiah tanpa merusak database berkat fitur Prepared Statements PDO.
• **Anti Cross-Site Scripting (XSS):** Coba masukkan teks: <script>alert('Halo')</script> pada nama lengkap. Hasilnya: Tidak memicu popup JavaScript melainkan muncul sebagai teks biasa berkat htmlspecialchars().

--- Selamat Belajar & Mempraktikkan Siklus Lengkap CRUD, MVC, OOP & Keamanan Web Dasar ---